FPGA Offloading Survey for TRACCC

Weekly Progress Report

Hao-Chun, Liang

Master's Thesis Progress Report

January 14, 2026

Advisor: Prof. Bo-Cheng, Lai

Completed Work Overview

	Task	Meaning
✓	FP32 vs FP64 Validation	Confirms FPGA can use native single-precision without physics degradation
✓	NCU Profiling Analysis	Identifies GPU bottlenecks (register pressure, cache misses, warp stalls)
✓	Nsys Profiling Analysis	Quantifies kernel time distribution across the pipeline
✓	FPGA Suitability Assessment	Determines which kernels are good FPGA candidates
✓	Alveo V80 Resource Estimation	Estimates DSP58 utilization and parallel pipeline count
✓	PCIe Latency Analysis	Measures data transfer overhead for GPU-FPGA communication

Output: `doc/survey-fpga.md` (3000+ lines of documented analysis)

1. FP32 vs FP64 Precision Validation

What I did:

- Built TRACCC with `-DTRACCC_CUSTOM_SCALARTYPE=float` and `=double`
- Ran Kalman fitter tests on 10,000 muon tracks (1 GeV, 10 GeV configurations)
- Compared pull distributions (μ, σ) and χ^2/NDF

What it means:

- **Result:** FP32 and FP64 produce **identical** physics results
- Pull σ differs by < 0.003 (within statistical noise)
- $\chi^2/\text{NDF} = 1.0042 \pm 0.0020$ for both precisions
- **Implication:** DSP58 native FP32 is safe for FPGA offloading — no precision loss

2. NCU Profiling Analysis

What I did:

- Profiled `propagate_to_next_surface` kernel with NVIDIA Nsight Compute
- Analyzed register pressure, cache hit rates, warp stall reasons

What it means:

- **93% warp stall cycles** — kernel is latency-bound, not compute-bound
- **96–128 registers/thread** — high register pressure limits occupancy
- **46–54% L1 cache hit** — unpredictable memory access patterns
- **Implication:** GPU's SIMT model is fundamentally inefficient for this workload. FPGA's pipelined execution can eliminate stalls.

3. Nsys Profiling Analysis

What I did:

- Ran full pipeline with Nsight Systems to measure kernel time distribution
- Analyzed 2,144 kernel invocations across 100 events

What it means:

Kernel	GPU Time	Implication
propagate_to_next_surface	63.0%	Primary FPGA target
build_tracks	14.6%	Must stay on GPU (pointer chasing)
Other kernels	22.4%	Mostly FPGA-suitable
Total FPGA-suitable	**~80%**	

Implication: 80% of GPU work could potentially move to FPGA.

4. FPGA Suitability Assessment

What I did:

- Analyzed each kernel's computational pattern (MAC chains, memory access, branching)
- Mapped operations to DSP58 capabilities

What it means:

Good for FPGA:

- RK4 propagation (MAC chains)
- B-field polynomial (Horner's method)
- Matrix-vector ops (systolic arrays)
- CCL clustering (well-known FPGA pattern)

Keep on GPU:

- Track deduplication (irregular branching)
- build_tracks (pointer chasing)
- 6×6 matrix inversion (complex control)

Implication: Clear partitioning strategy — offload compute-regular kernels, keep irregular ones on GPU.

5. Alveo V80 Resource Estimation

What I did:

- Estimated DSP58 usage per track pipeline ($\sim$110 DSP58)
- Calculated parallel pipeline capacity from V80 specs (10,848 DSP58)

What it means:

Resource	Estimate
DSP58 per pipeline	$\sim$110 (RK4 + matrix ops)
Parallel pipelines	$\sim$98 (10,848 / 110)
HBM capacity	32 GB (sufficient for geometry + B-field)

Implication: V80 can theoretically run $\sim$98 tracks in parallel with deeply pipelined execution. This is a **theoretical estimate** — actual numbers need Vitis HLS synthesis.

6. PCIe Latency Analysis

What I did:

- Measured GPU↔Host transfer latency as proxy for GPU↔FPGA
- Tested different data sizes (params only vs full state vs + Jacobians)

What it means:

Transfer Scenario	Overhead	Assessment
Parameters only (24 B/track)	2.7%	✓Acceptable
Full track state (176 B/track)	13%	Concerning
+ Jacobians (320 B/track)	18%	✗Prohibitive

Implication: Must minimize data transfer. Cache covariance on FPGA HBM, transfer only 24-byte parameter vectors.

Next Step: Validate Per-Step Sync Barrier

Critical Blocker

CKF algorithm requires GPU↔FPGA synchronization at **each of 15 surfaces**. If sync overhead $> 200\mu$s/step, FPGA offloading is **not viable**.

What needs to be done:

1. Instrument CKF loop to measure actual per-step synchronization time
2. Test with GPU↔Host sync as proxy (no FPGA hardware yet)
3. Calculate total overhead: $15 \times$ sync_time vs 23ms baseline

Decision criteria:

- If overhead $< 5\%$ ($<$1.15ms total): Proceed with FPGA development
- If overhead 5–15%: Consider async/overlap strategies
- If overhead $> 15\%$: FPGA offloading not viable for CKF

This is the go/no-go decision point before any FPGA development.

Summary

Completed:

1. FP32 = FP64 for physics $\Rightarrow$ DSP58 native FP32 is safe
2. 63% GPU time in one kernel with 93% stalls $\Rightarrow$ ideal FPGA candidate
3. 80% of work is FPGA-suitable $\Rightarrow$ clear partitioning strategy
4. V80 can support $\sim$98 parallel pipelines $\Rightarrow$ sufficient resources
5. PCIe overhead 2.7% for params-only $\Rightarrow$ transfer strategy defined

Next:

- Measure per-step sync barrier overhead (critical go/no-go)
- If viable: prototype RK4 kernel in Vitis HLS

Full documentation: `doc/survey-fpga.md`